FH Salzburg
MultiMediaTechnology

Basics Game Development

Input Handling, Animations

Technology
Health
Media

Input

Input Devices

Input Methods

Classic input methods

Button
(Keyboard, Buttons, D-Pad)

1-axis controller
(Steering wheel)

2-axis controller
(Joystick, Mouse, Analog stick …)

Motion input methods

Motion controller
(Wii Remote, Playstarion DualShock, Smartphone, VR, …)

Motion tracking
(Kinect, Camera, Sensors, …)

Input Mapping

- **User customization**
  - Player want to set the controls manually
- **Cross-platform**
  - XBox: Button A
  - PS5: Button X
  - Nintendo: Button B
- **Solution**
  - Define „**virtual inputs**“ and map it to hardware input.
    - Virtual event: „Jump“
    - PC hardware input: „SPACE“
    - PS5 input: „Button X“
    - …

Remappable Controls Are Expected by Your Players!

Input in SFML

- **SFML input support:**
  - Mouse
  - Keyboard
  - Joystick / gamepad
  - Sensors
  - Touch
- Popular 3rd party OIS-solutions (OIS = object oriented input system):
  - SharpInputSystem

Getting the Device State

- **Polling**
  - Constantly query the device to find out it‘s state.

```
if (Keyboard.IsKeyPressed(Keyboard.Key.Space))
    Console.WriteLine($"Key {Keyboard.Key.Space} pressed");
```

- **Callbacks**
  - Register callback which is triggered when a input device is changed.

```
window.KeyPressed += OnKeyPressed;
```

```
private void OnKeyPressed(object? sender, KeyEventArgs e)
{
    if (e.Code == Keyboard.Key.Space)
        Console.WriteLine($"Key {e.Code} pressed");
}
```

Input Manager

- **In games both methods are common.**
  - Player Controls -> Polling
  - User interface -> Callbacks
- **Goals**
  - Provide simple access to input states.
    - Caches the state of the devices for fast polling
  - Support remapping of events.
  - Support multiple players.
  - Can be implemented as Singleton

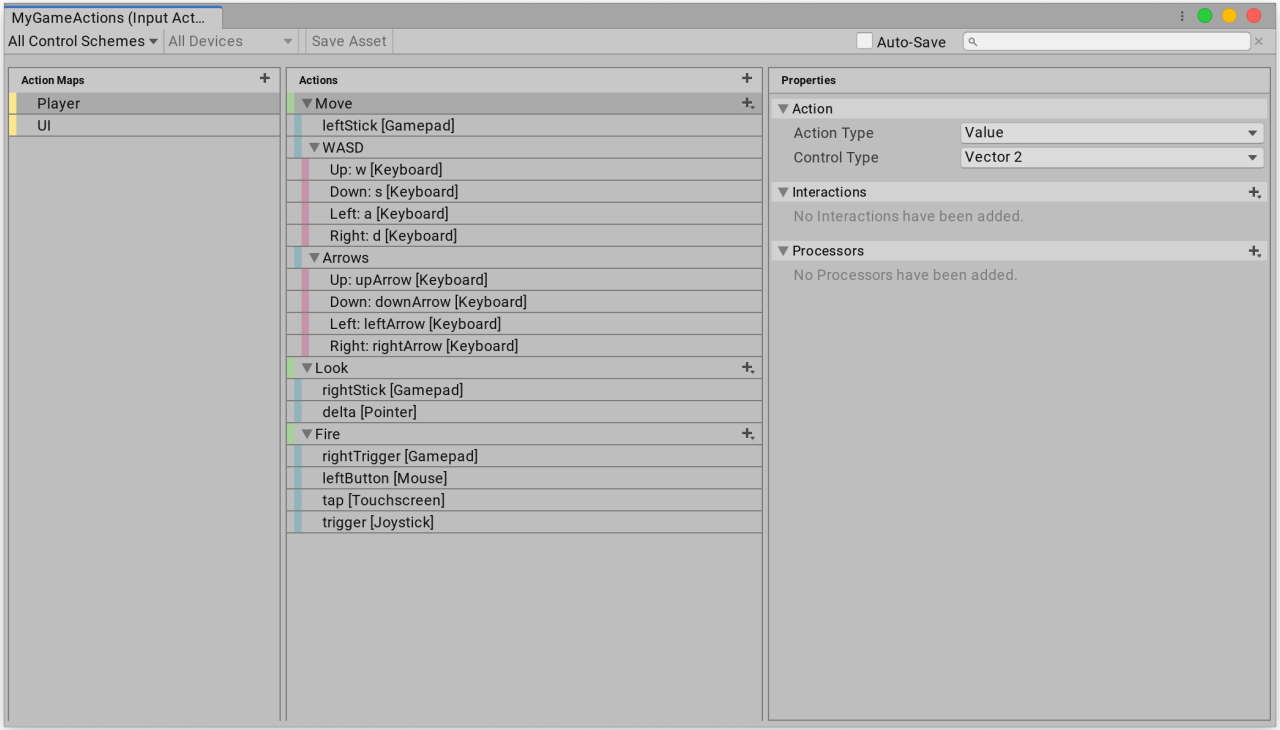
Input Manager Singleton

```
InputManager

+ static getInstance()

+ isKeyDown(keyCode:int)
+ isKeyUp(keyCode:int)
+ isKeyPressed(keyCode:int)
+ isKeyReleased(keyCode:int)

+ isKeyDown(action:string, playerIdx:int)
+ isKeyUp(action:string, playerIdx:int)
+ isKeyPressed(action:string, playerIdx:int)
+ isKeyReleased(action:string, playerIdx:int)

+ bind(name:string, keyCode:int, playerIdx:int)
+ unbind(name:string, playerIdx:int)
```

```
1 reference
void Initialize(){
    InputManager.GetInstance().Bind("jump", Keyboard.Key.Space, 0);
}
0 references
void Update() {
    bool isKeyReleased = InputManager.GetInstance().IsKeyReleased("jump", 0);
}
```

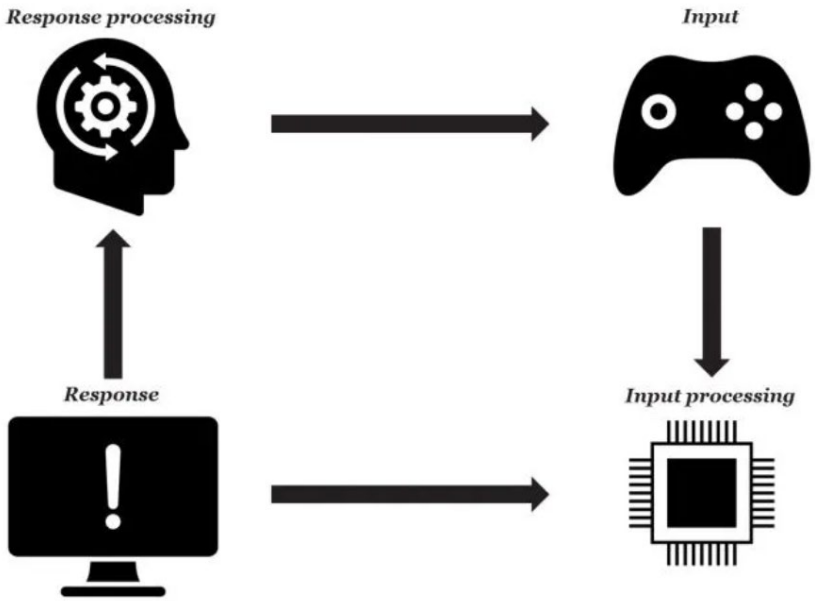
Side Note: Cache your Singlenton references instead accessing it every time through GetInstance. (it‘s faster)

Unity Input System

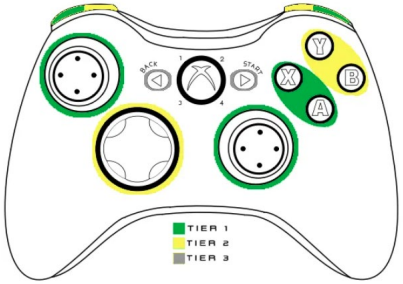


Controls - Action Cycle

- **The player sends the input signal, then this signal is processing by the system, the player sees the response on the screen, process this response and sends new input signal.** All this creates an ongoing **correction cycle** (you can find out more about the theory behind that in Donald Norman's "Human action cycle").
- **Good controls should not break the correction cycle and don't frustrate the player on any of interaction stages.**



Controls - Accessibility



- **Hand Limitations**
  - Three main “finger groups”
- **<u>Fitt‘s Law</u> (for our case)**
  - the fewer the distance to the button and the bigger the button, the more accessible the button is.

The most frequent actions should be in the most accessible places and match primary control group of the player’s hand.

Animation

Historical Animation

- Animation comes from the Latin word animatus, meaning **“to give life”** or “to live”.
- In traditional animation, the artist had full control of every object location and velocity across the scene for every frame in the sequence.
- The game animator must give up that control, however, in order to allow players to feel that they are driving the actions of the game.

- **Graphics programmer:**
  - Acts as a link between the artist and the player
  - Must understand:
    1. mechanics of the game
    2. mechanics of animation

Cel Animation

- The term *cel*, short for *celluloid*, refers to the transparent sheet used by film animators to draw images that would later be placed on top of a generally static background and photographed as an individual frame on film.
- Same conceptpual process we use when animating sprites (sprite animation).

- **Basic steps:**
  1. Add code to track the animation sequence
  2. Loop through the animation
  3. Render the appropriate frame from the sprite sheet.

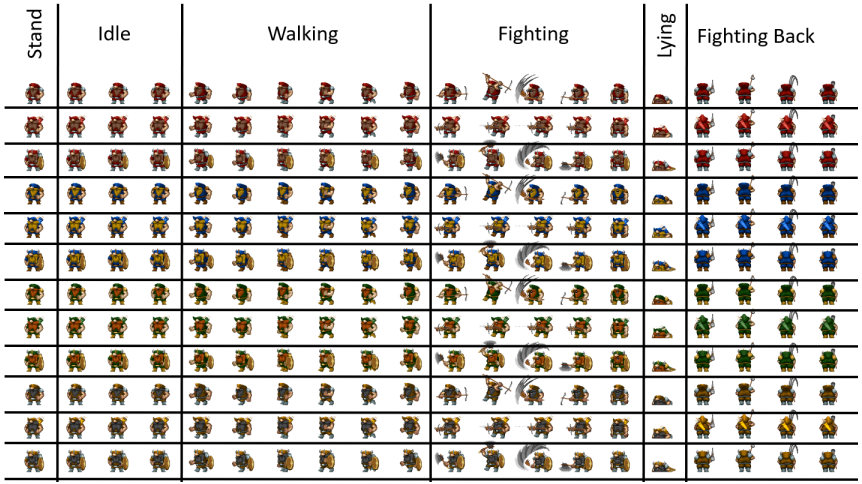
Animation Cycles

- In a production environment, you have multiple animation cycles per character.
- This might include cycles for running, attacking, jumping, blocking, and even just standing around.
- You need to create a **robust animation system that will allow you to switch between these cycles.**

AliEnd: selection of Newman‘s various animation cycles.

Sprite Sheet

- **Compact and efficient storage**
  - **Needs meta data** - which frame belongs to which animation.
- **Multiple Animations**
  - Store dictionary of animations with start and end frame.
  - Update and render only the current active animation.



Timing

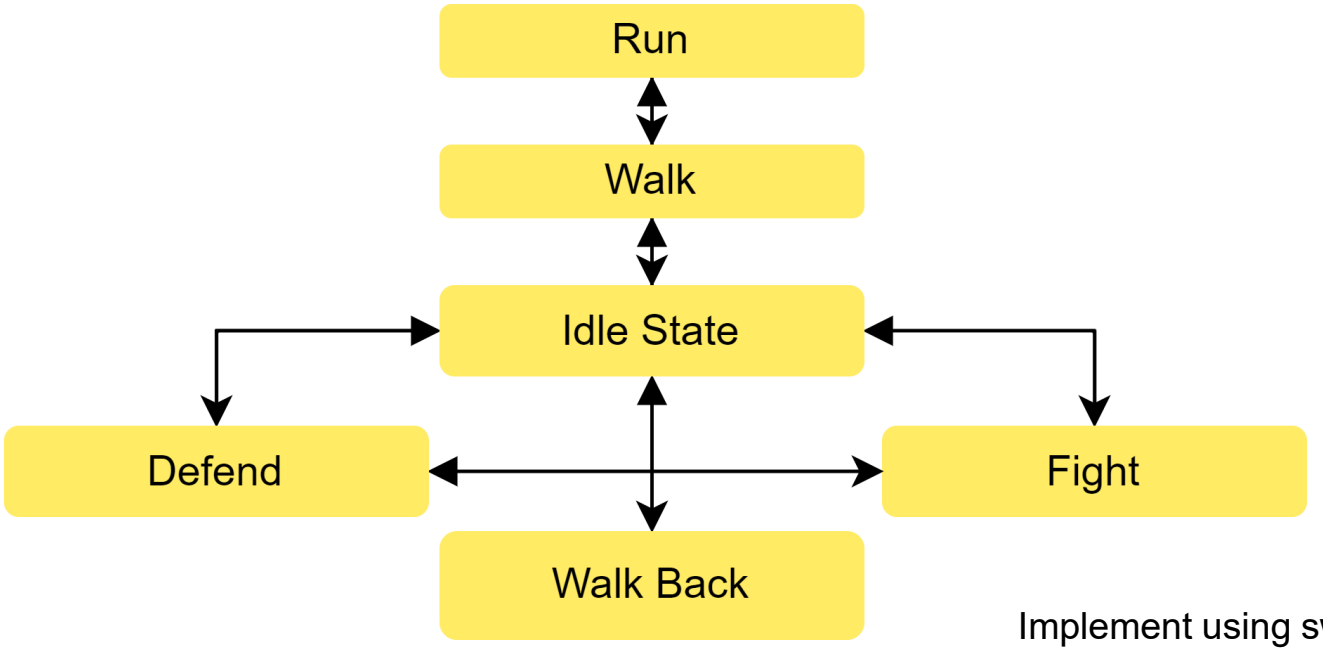
- Animations are usually played at a fixed frame rate. The designer of the animation chooses it so that the movements look smooth. The following criteria are considered:
  - How fast is the object moving?
  - How big is the object?

Smaller and slower-moving objects will require a lower framerate than larger, fast objects for smooth-looking movement.

Goals of Animation

- **Timing**
- **Slow In / Slow Out**
  - concept of acceleration applied to animation
- **Arcs**
  - The principle of arcs simply states that things in the real world tend to move in arcs.
- **Follow-Through and Overlapping Action**
  - “Observation of Newton's First Law”

Animation State Machine

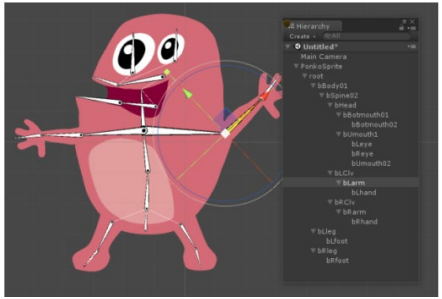


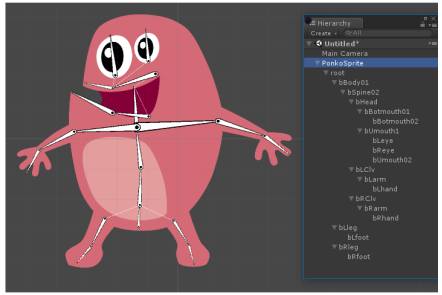
Implement using switch statement or the “State pattern”

Issues of Sprite Animation

- Data size of sprite sheets (>24 frames/second)
- Frames cannot be interpolated
  - No smooth transition / interpolation from one state to another
- A lot of 2d games use 2d **skeletal animation** (aka **bones animation**) instead.

- https://unity.com/features/2danimation





Comparison

	FRAME ANIMATION	SKELETAL ANIMATION
Implementation	Simple	Complex
Performance	Super fast	Rather fast
Data	Huge	Small
Production	Easy	Needs more setup
Animation Smoothness	Limited to frame count	Smooth
Animations transition	Hard	Smooth
Character Variation	Difficult	Flexible
Animation Attachment	No	Object can be attached to bone (e.g., weapon)

FH Salzburg
MultiMediaTechnology

Basics Game Development

Input Handling, Animations - End

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at